Analysis of National/International Survey Data

Contents

1. Introduction to Complex Sample Analysis . . . 2
    1.1. Simple Random Sampling vs. Complex Sampling . . . 2
    1.2. Three Common Features of Complex Sampling . . . 2
    1.2.1. Clustering . . . 2
    1.2.2. Stratification . . . 2
    1.2.3. Post-stratification (Weighting) . . . 3
    1.3. Why Complex Sample Analysis Software? . . . 3
2. `{EdSurvey}` Basics . . . 3
    2.1. Overview of `{EdSurvey}` . . . 4
    2.2. Setting up Environment . . . 4
    2.3. Downloading Data . . . 4
    2.4. Reading in Data . . . 5
    2.5. Getting to Know the Data Format . . . 5
    2.5.1. Retrieving Survey Weights . . . 6
    2.5.2. Retrieving Stratum and PSU Variables . . . 6
    2.6. Explore Variable Distributions . . . 7
    2.7. Retrieving Data for Manipulation . . . 8
    2.7.1. Retrieving a Set of Variables . . . 8
    2.7.2. Recoding Variables . . . 8
    2.7.3. Applying Attributes . . . 10
3. `{EdSurvey}` Analytics . . . 10
    3.1. Correlation . . . 10
    3.1.1. Weighted Correlation . . . 10
    3.1.2. Unweighted Correlation . . . 12
    3.2. Tabulation . . . 13
    Subsetting Data . . . 15
    3.3. Linear Regression . . . 16
    3.4. MANCOVA . . . 17
    3.5. Logistic Regression . . . 18

3.5.1. Odds Ratio 19
3.5.2. Wald Test 19
4. Summary 19

1. Introduction to Complex Sample Analysis

Objectives

- Understand the difference between simple random sample and complex sample.
- Understand common features of complex sampling.
- Understand the purpose of using complex sample analysis software.

1.1. Simple Random Sampling vs. Complex Sampling

Most statistical analyses assume that the data collected are from a **simple random sample** of the population. So say, for example, that you are conducting a study of K-12 school teachers in Texas (this is the *"population"*), a simple random sample would be where all teachers in Texas (or *"sampling units"*) are equally likely to be sampled.

However, it's not always possible to take a simple random sample. Simple random sampling (SRS) requires access to the whole population of interest, which may not be feasible for large populations. If sampling units widely spread out geographically, it may also be prohibitively expensive to access and sample across the whole area. Or, if some members of the population (e.g., of a particular demographic background) are relatively low in number, a simple random sample may not obtain enough (or any) of these individuals to reliably measure their responses. So, even if a random sampling design is available, it may be much cheaper or more efficient to use a complex sampling design, such as multi-stage sampling, clustering, stratification, etc.

In a **complex sample**, members of the population don't have the same probability of being selected into the sample. Complex samples are most often used for surveys, especially large *national or international* ones where simple random sampling is simply not practical.

So, what are the most common approaches of complex sampling and why and when are they used?

1.2. Three Common Features of Complex Sampling

1.2.1. Clustering

In **cluster sampling**, the population is split into similar groups of individuals (*"clusters"*) and then a sample of these clusters is taken, so that all of the elements in the selected clusters are included in the sample. Clustering is appropriate when we expect elements in different clusters to be relatively similar ("homogeneous").

For example, you may cluster teachers according to their schools and then select a few schools (surveying all teachers within those selected schools) to obtain a clustered sample of K-12 teachers in Texas.

1.2.2. Stratification

Stratified sampling involves splitting members of the population into subgroups (*"strata"*) before sampling, and then applying sampling (usually SRS) separately within each and every group (*"stratum"*). This is in contrast to cluster sampling where whole clusters are sampled, rather than samples of individuals being

taken within each group. Stratified sampling is appropriate when elements in different groups are relatively dissimilar ("heterogeneous").

For example, you may stratify schools together according to their type (e.g., public, private, charter, etc.) and then take a sample of teachers within the same school type, to ensure that teachers from each school type are adequately represented in the sample.

Each stratum can be sampled in proportion to the relative size of that subgroup in the total population (*"proportionate allocation"*) to make the overall sample as representative as possible. Or, larger samples can be obtained in the strata with greater variability to minimize the sampling variance (*"optimum allocation"*), improving the efficiency of the sample overall.

It is also possible to combine stratified sampling with cluster sampling. For example, you may stratify schools by type and then take cluster samples of schools within each stratum. This is an example of *one-stage cluster sampling* but further stages of sampling could be also included. In *two-stage cluster sampling*, for example, after selecting school clusters, you may interview only a sample of the teachers in each selected school.

1.2.3. Post-stratification (Weighting)

After completing your survey, you may find that the sample you have taken is *not* representative of the population. For example, 80% of K-12 teachers in Texas are female whereas only 60% of your sample are female, so that females are *under-sampled* in your study. Such differences can be due to non-response or incomplete coverage, which are an inevitable consequence of the fact that we cannot sample everyone in the population nor compel them to respond. In such case, **sampling weights** can be calculated to post-stratify the sample, to ensure that the results are representative of the population.

1.3. Why Complex Sample Analysis Software?

The most important thing to understand about complex sampling is that a more sophisticated analysis is needed when analyzing the data collected — standard approaches are not necessarily appropriate. You must take account of the sampling design in order for your conclusions to be reliable, whether you are estimating a characteristic of the population or testing for effects.

The sampling design affects the calculation of both **point estimates** and **standard errors** of the estimates. If you ignore the sampling design (e.g., when you assume simple random sampling but actually another type of sampling design was used), the standard errors will likely be underestimated, possibly leading to results that seem to be statistically significant when in fact they are not (*"false positives"*).

In SPSS, complex sample analysis can be generated, when used alongside the Analyze » Complex Samples menu, to ensure that the sampling design is incorporated into the analysis.

In R, the `{survey}` package similarly allows you to specify a complex survey design and carry out appropriate analyses taking the design into account. Other packages in R, such as the `{anesrake}` package, are also useful for implementing survey weighting.

2. `{EdSurvey}` Basics

Objectives

- Understand the basic functions of the `{EdSurvey}` package.
- Demonstrate how to use the package for manipulating the Early Childhood Longitudinal Survey, Kindergarten Class of 2010-11 (ECLS-K:2011) data.

2.1. Overview of {EdSurvey}

The {EdSurvey} package provides functions to perform analyses that account for complex sampling designs, such as clustering, stratification, weighting, etc. The package also seamlessly takes advantage of the {LaF} package to read in data only when required for an analysis.

2.2. Setting up Environment

```
### Set the 'Cloud' mirror as a default CRAN (Comprehensive R Archive Network) repository
options(repos="https://cloud.r-project.org")

### Install and load the package
install.packages("EdSurvey")
library(EdSurvey)
```

2.3. Downloading Data

You can use the package's download functions to acquire the survey data of interest, including the following:

- ECLS-K: Early Childhood Longitudinal Study, Kindergarten Class of 1998–99 and Early Childhood Longitudinal Study, Kindergarten Class of 2010–11 (downloadECLS_K).
- ELS: Education Longitudinal Study (downloadELS)
- HSLS: High School Longitudinal Study (downloadHSLS)
- PISA: The Programme for International Student Assessment (downloadPISA)
- TIMSS: Trends in International Mathematics and Science Study and TIMSS Advanced (downloadTIMSS, downloadTIMSSAdv)
- PIRLS: Progress in International Reading Literacy Study (downloadPIRLS).
- ePIRLS: Electronic Progress in International Reading Literacy Study (download_ePIRLS)
- CIVED: The Civic Education Study 1999 and International Civic and Citizenship Study (downloadCivEDICCS)
- ICILS: International Computer and Information Literacy Study (downloadICILS)
- PIAAC: Programme for the International Assessment of Adult Competencies (downloadPIAAC)
- TALIS: Teaching and Learning International Survey (downloadTALIS)
- NAEP: National Assessment of Educational Progress (downloadNAEP)

The following example shows how to download the ECLS-K:2011 public-use data file.

```
downloadECLS_K(years = 2011, root = "C:/Users/Jason Lee/OneDrive - Texas Tech University/files/ttu/educa

### For macOS users
# downloadECLS_K(years = 2011, root = "~\")
```

The downloadECLS_K() function creates a new folder "ECLS_K" and a subfolder "2011" in the directory that you specified and then saves the survey data into the "2011" folder.

[Note] Beginning for the ECLS_K:2011 Study, the ChildK5p.zip source data file is a DEFLATE64 compressed zip file. This means that you must manually extract the childK5p.dat file using an external zip program capable of handling DEFLATE64 zip format. As existing R functions are unable to handle this zip format natively. When unzipping, choose the "Extract Here" option so that the "2011" folder contains the following two files:

- childK5p.dat
- ECLSK2011_K5PUF.sps

2.4. Reading in Data

The next step is to read in the data. For each study, there is a `read` function to assist in reading and processing its data. The read functions are as follows:

- ECLS-K: Early Childhood Longitudinal Study, Kindergarten Class of 1998–99 (`readECLS_K1998`)
- ECLS-K: Early Childhood Longitudinal Study, Kindergarten Class of 2010–11 (`readECLS_K2011`)
- ECLS-B: Early Childhood Longitudinal Study, Birth Cohort (`readECLS_B`)
- ELS: Education Longitudinal Study (`readELS`)
- HSLS: High School Longitudinal Study (`readHSLS`)
- PISA: The Programme for International Student Assessment (`readPISA`)
- TIMSS: Trends in International Mathematics and Science Study and TIMSS Advanced (`readTIMSS`, `readTIMSSAdv`)
- PIRLS: Progress in International Reading Literacy Study (`readPIRLS`)
- ePIRLS: Electronic Progress in International Reading Literacy Study (`read_ePIRLS`)
- CIVED: The Civic Education Study 1999 and International Civic and Citizenship Study (`readCivEDICCS`)
- ICILS: International Computer and Information Literacy Study (`readICILS`)
- PIAAC: Programme for the International Assessment of Adult Competencies (`readPIAAC`)
- TALIS: Teaching and Learning International Survey (`readTALIS`)
- NAEP: National Assessment of Educational Progress (`readNAEP`)

The `readECLS_K2011()` function creates an `edsurvey.data.frame` that stores information about the data, including the format/layout of the data as well as all variables at multiple data levels: student, teacher, school, and/or home. The following example shows how to load the ECLS-K:2011 public-use data.

```
eclsk11 <- readECLS_K2011("C:/Users/Jason Lee/OneDrive - Texas Tech University/files/ttu/education/class

### For macOS users
# eclsk11 <- readECLS_K2011("~/ECLS_K/2011")
```

The function may take several minutes to run the first time, but subsequent calls to the function are stored on your drive for easy access and near instant retrieval. Once read in, the data from different data files can be merged and analyzed.

2.5. Getting to Know the Data Format

To get general information about the data, simply type the name of the `data.frame` object (i.e., `eclsk11`).

```
eclsk11
```

Some of the basic functions, such as `dim()` and `colnames()`, also work on an `edsurvey.data.frame`.

```
dim(eclsk11)
```

```
## [1] 18174 26060
```

```
# colnames(eclsk11)
```

To conduct a more powerful search of variables, use the `searchSDF()` function, which returns variable names and labels from an `edsurvey.data.frame`. Let's use the variable "a4useaubk" as an example.

```
searchSDF("a4useaubk", eclsk11, levels = TRUE)
```

```
## Variable: a4useaubk
## Label: A4 C1K READ USING READ-ALONG/AUDIO BOOKS
## Levels (Lowest level first):
##      1. 1: NEVER OR HARDLY EVER
##      2. 2: ONCE OR TWICE A MONTH
##      3. 3: ONCE OR TWICE A WEEK
##      4. 4: ALMOST EVERY DAY
##      -9. -9: NOT ASCERTAINED
```

Access a full codebook using the `showCodebook()`function. This function pairs well with the `View()` function to more easily explore the data.

```
View(showCodebook(eclsk11))
```

2.5.1. Retrieving Survey Weights

The variables associated with each **survey weight** (105 different weights in total) can be seen from the `showWeights()` function.

```
showWeights(data = eclsk11, verbose = FALSE)
```

Here we hide the (lengthy) results, but you can easily see them by running the same code. Selection of survey weights is especially important for the ECLS-K:2011 data. Once selected, the survey weight is specified using the `weightVar` argument in the `{EdSurvey}` analytic functions. For example, the weight `w9c29p_9t90` applies to base-year (2010-11) respondents who had student data from Kindergarten and fifth grade and parent interview data from fall or spring kindergarten and fifth grade.

Consult the ECLS-K:2011 user manual for guidance on choosing the most appropriate sample weight for your analysis.

2.5.2. Retrieving Stratum and PSU Variables

The functions `getStratumVar()` and `getPSUVar()` return the default **stratum** and **PSU** (ID) variables associated with a weight variable, respectively. For example, the total student weight from the 9th round (at 5th grade) `weightVar = "w9c29p_9t90"` returns the following:

```
getStratumVar(data = eclsk11, weightVar = "w9c29p_9t90")
```

```
## [1] "w9c29p_9t9str"
```

```
getPSUVar(data = eclsk11, weightVar = "w9c29p_9t90")
```

```
## [1] "w9c29p_9t9psu"
```

2.6. Explore Variable Distributions

The `summary2()` function produces *weighted* and *unweighted* descriptive statistics for a variable. By default, estimates are not weighted. For example, the variable `x9povty_i` ("Imputed Poverty Level") returns the following output:

```
summary2(eclsk11, "x9povty_i")
```

```
## Estimates are not weighted.
##                                                                    x9povty_i    N
## 1                                                                  (Missing) 7954
## 2                                                 1: BELOW POVERTY THRESHOLD 2185
## 3 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD 2226
## 4                           3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD 5809
##    Percent
## 1 43.76582
## 2 12.02267
## 3 12.24827
## 4 31.96324
```

Note that by default, the `summary2()` function includes omitted levels. To remove those, set `omittedLevels = TRUE`:

```
summary2(eclsk11, "x9povty_i", omittedLevels = TRUE)
```

```
## Estimates are not weighted.
##                                                                    x9povty_i    N
## 1                                                 1: BELOW POVERTY THRESHOLD 2185
## 2 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD 2226
## 3                           3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD 5809
##    Percent
## 1 21.37965
## 2 21.78082
## 3 56.83953
```

When specified in the argument `weightVarThe`, the `summary2` function returns the weighted number of cases, the weighted percent, and the weighted standard error for a categorical variable. For example, using the total student weight from the 9th round `weightVar = "w9c29p_9t90"`:

```
summary2(eclsk11, "x9povty_i", weightVar = "w9c29p_9t90")
```

```
## Warning in calcEdsurveyTable(formula, data, weightVar, jrrIMax,
## pctAggregationLevel, : Removing 10214 rows with 0 weight from analysis.
```

```
## Estimates are weighted using the weight variable 'w9c29p_9t90'
##                                                                    x9povty_i    N
## 1                                                                  (Missing)    0
## 2                                                 1: BELOW POVERTY THRESHOLD 1589
## 3 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD 1692
## 4                           3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD 4679
##   Weighted N Weighted Percent Weighted Percent SE
## 1        0.0          0.00000                  NA
## 2   885855.4         22.48863           0.9229348
## 3   930036.4         23.61023           0.6935488
## 4  2123233.2         53.90114           1.0799958
```

2.7. Retrieving Data for Manipulation

2.7.1. Retrieving a Set of Variables

The `getData()` function can extract a set of variables for data manipulation and analysis. For example, let's create a data set `dat0` that contain the variables `x_chsex_r` ("Sex of Students"), `p5sumsch` ("Child attended summer school" at 2nd grade), and `p5nhrprm` ("Hours per day child attended summer school" at 2nd grade), as well as a 2nd-grade weight variable `w5cf5pf_50`.

```
dat0 <- getData(data = eclsk11, varnames = c("x_chsex_r", "p5sumsch", "p5nhrprm", "w5cf5pf_50"), omitte
```

By default, setting `omittedLevels = TRUE` removes special values such as NAs. The `getData()` function tries to help by dropping the levels of factors for regression, tables, and correlations that are not typically included in analyses. Here we set `omittedLevels = FALSE` in order to recode special values in an example to follow.

The argument `addAttributes = TRUE` ensures that the analysis functions shown so far can continue to be used with the resulting data set `dat0`.

2.7.2. Recoding Variables

Example 1 Data recording is of particular importance when performing analyses with the ECLS:K-2011 data given the complexity of the survey design. Suppose that you are interested in exploring student performance in Mathematics based on the number of hours/day a parent reported their child attended summer school.

The `table()` function is a simple way of ascertaining the number of values for each level of a variable. Using this function for the variable `p5sumsch` indicates that 449 parents reported their child attending summer school. Also, using this function for the variable `p5nhrprm` indicates that if attended, children spent 2 to 7 hours per day.

```
table(dat0$p5sumsch, useNA = "ifany")
```

```
##
##              1: YES                2: NO      -8: DON'T KNOW -9: NOT ASCERTAINED
##                 449                 3913                    1                   7
##                <NA>
##               13804
```

```
table(dat0$p5nhrprm, useNA = "ifany")
```

```
##
##     2     3     4     5     6     7  <NA>
##    55    72   126    43    87    62 17729
```

In order to include (3,913) children who did not attend summer school into the analysis – those that were skipped by the design of the survey, you need to recode `p5nhrprm` values (NA) to 0 where `p5sumsch` = "2: NO".

```
dat0$p5nhrprm <- ifelse(dat0$p5sumsch == "2: NO", 0, dat0$p5nhrprm)
table(dat0$p5nhrprm, useNA = "ifany")
```

```
##
##     0     2     3     4     5     6     7  <NA>
##  3913    55    72   126    43    87    62 13816
```

Note that the 3,913 cases of `p5sumsch` = "2: NO" are now included as a level (0) in the recoded `p5nhrprm`.

Example 2 A second example of recoding a variable in response to a skip pattern pertains to the frequency that (a) a child does homework (`p9hmwork`) and (b) a parent/someone else helps (`p9hlphwk`) in the 5th-grade data. The `levelsSDF()` function is useful to show a variable's levels and their unweighted n-sizes.

```
levelsSDF("p9hmwork", eclsk11)
```

```
## Levels for Variable 'p9hmwork' (Lowest level first):
##     1. 1: NEVER (n=294)
##     2. 2: LESS THAN ONCE A WEEK (n=381)
##     3. 3: 1 TO 2 TIMES A WEEK (n=1406)
##     4. 4: 3 TO 4 TIMES A WEEK (n=4236)
##     5. 5: 5 OR MORE TIMES A WEEK (n=3838)
##     -1. -1: NOT APPLICABLE* (n=51)
##     -7. -7: REFUSED* (n=1)
##     -8. -8: DON'T KNOW* (n=13)
##     -9. -9: NOT ASCERTAINED* (n=0)
##     NOTE: * indicates an omitted level.
```

```
levelsSDF("p9hlphwk", eclsk11)
```

```
## Levels for Variable 'p9hlphwk' (Lowest level first):
##     1. 1: NEVER (n=643)
##     2. 2: LESS THAN ONCE A WEEK (n=1786)
##     3. 3: 1 TO 2 TIMES A WEEK (n=3774)
##     4. 4: 3 TO 4 TIMES A WEEK (n=2558)
##     5. 5: 5 OR MORE TIMES A WEEK (n=1094)
##     -1. -1: NOT APPLICABLE* (n=359)
##     -7. -7: REFUSED* (n=2)
##     -8. -8: DON'T KNOW* (n=4)
##     -9. -9: NOT ASCERTAINED* (n=0)
##     NOTE: * indicates an omitted level.
```

The skip pattern for this sequence of survey questions is as follows: If `p9hmwork` = "1: NEVER" then `p9hlphwk` is skipped and coded "-1: NOT APPLICABLE". To include this subset of children in the analysis, `p9hlphwk` values (NA) need to be recoded to 0. First, retrieve the data via the `getData()` function (along with a few other variables for a subsequent example) to recode using an `ifelse` statement:

```
dat1 <- getData(data = eclsk11, varnames = c("p9hmwork", "p9hlphwk", "x_chsex_r", "x9rscalk5", "x9mscall

dat1$p9hlphwk <- ifelse(dat1$p9hmwork == "1: NEVER" &
                        dat1$p9hlphwk == "-1: NOT APPLICABLE", 0,
                        dat1$p9hlphwk)
```

Then, use the `table()` function to view the counts of each level.

```
table(dat1$p9hlphwk, useNA = "ifany")
```

```
##
##    0    1    2    3    4    5    6    7    8 <NA>
##  294  643 1786 3774 2558 1094   65    2    4 7954
```

Note that the 294 cases of `p9hmwork` = "1: NEVER" are now included as a level (0) in the recoded `p9hlphwk`.

2.7.3. Applying Attributes

A helper function that pairs well with `getData()` is `rebindAttributes()`. This function allows you to reassign the attributes from a survey data set to a data frame that might have had its attributes stripped during the manipulation process. Once attributes have been rebinded, all variables (including those outside the original data set) are available for use in the `{EdSurvey}` analytic functions.

For example, the survey attributes from the `eclsk11` data are applied to the data frame `dat1` created in the second example above.

```
dat1 <- rebindAttributes(dat1, eclsk11)
```

3. {EdSurvey} Analytics

Objectives

- Understand the analytic functions of the `{EdSurvey}` package.
- Demonstrate how to use the package for analyzing the Early Childhood Longitudinal Survey, Kindergarten Class of 2010-11 (ECLS-K:2011) data.

3.1. Correlation

The `{EdSurvey}` package provides multiple correlation methods that fully account for the complex sampling design. They include:

- **Pearson product-moment** correlation for two continuous variables
- **Spearman rank** correlation for two ranked variables
- **Polyserial** correlation for a categorical variable and a continuous variable
- **Polychoric** correlation for two categorical variables

3.1.1. Weighted Correlation

In the following example, the variables `x9mscalk5` ("Mathematics IRT scale") and `x9povty_i` ("Imputed Poverty Level") and the total student weight `w9c29p_9t90` are read in to calculate Pearson correlation:

```
cor_pearson <- cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method = "Pearson", weightVar =
```

```
## Converting "x9povty_i" to a continuous variable.
```

```
## Warning in cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method =
## "Pearson", : Removing rows with 0 weight from the analysis.
```

```r
cor_pearson
```

```
## Method: Pearson
## full data n: 18174
## n used: 7905
##
## Correlation: 0.3720479
## Standard Error: 0.01504248
## Confidence Interval: [0.3416126, 0.4017036]
##
## Correlation Levels:
##   Levels for Variable 'x9povty_i' (Lowest level first):
##     1. 1: BELOW POVERTY THRESHOLD
##     2. 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD
##     3. 3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD
```

Note that the warning message "Removing rows with 0 weight from analysis" indicates that this call has excluded rows with 0 weights from the calculation.

The `cor.sdf()` function features multiple correlation methods. The following example shows the differences in correlation coefficients among the Pearson, Spearman, and polychoric methods:

```r
cor_spearman <- cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method = "Spearman", weightVa
```

```
## Warning in cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method =
## "Spearman", : Removing rows with 0 weight from the analysis.
```

```r
cor_spearman
```

```
## Method: Spearman
## full data n: 18174
## n used: 7905
##
## Correlation: 0.3756259
## Standard Error: 0.01600371
## Confidence Interval: [0.3436635, 0.4075884]
##
## Correlation Levels:
##   Levels for Variable 'x9povty_i' (Lowest level first):
##     1. 1: BELOW POVERTY THRESHOLD
##     2. 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD
##     3. 3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD
```

```r
# cor_polychoric <- cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method = "Polychoric", we
# cor_polychoric
```

Changing the order of levels is useful when the levels are out of order or when you want to reverse the orientation of a series. Note that reordering levels will not change the magnitude of the correlation coefficient but only the direction —- from negative to positive in this example.

```r
cor_pearson_re <- cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method = "Pearson", weightVa
                          reorder = list(x9povty_i = c("3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD
                                                       "2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERC
                                                       "1: BELOW POVERTY THRESHOLD")))
```

```
## Converting "x9povty_i" to a continuous variable.
```

```
## Warning in cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method =
## "Pearson", : Removing rows with 0 weight from the analysis.
```

```r
cor_pearson_re
```

```
## Method: Pearson
## full data n: 18174
## n used: 7905
##
## Correlation: -0.3720479
## Standard Error: 0.01504248
## Confidence Interval: [-0.4017036, -0.3416126]
##
## Correlation Levels:
##   Levels for Variable 'x9povty_i' (Lowest level first):
##     1. 3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD
##     2. 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD
##     3. 1: BELOW POVERTY THRESHOLD
```

3.1.2. Unweighted Correlation

The `cor.sdf()` function also features the ability to calculate correlation without accounting for weights by setting `weightVar = NULL`.

```r
cor_pearson_unweighted <- cor.sdf(x = "x9mscalk5", y = "x9povty_i", data = eclsk11, method = "Pearson",
```

```
## Converting "x9povty_i" to a continuous variable.
```

```r
cor_pearson_unweighted
```

```
## Method: Pearson
## full data n: 18174
## n used: 9329
##
## Correlation: 0.3840037
## Standard Error: NA
## Confidence Interval: [NA]
##
## Correlation Levels:
##   Levels for Variable 'x9povty_i' (Lowest level first):
##     1. 1: BELOW POVERTY THRESHOLD
##     2. 2: AT OR ABOVE POVERTY THRESHOLD, BELOW 200 PERCENT OF POVERTY THRESHOLD
##     3. 3: AT OR ABOVE 200 PERCENT OF POVERTY THRESHOLD
```

3.2. Tabulation

Summary tables can be created using the `edsurveyTable()` function. For example, a table with the two variables `x_chsex_r` ("Sex of Students") and `p9curmar` ("Current Marital Status") shows the number and percentage of students by gender and their parent's current marital status. Percentages add up to 100 within each gender.

```
tbl1 <- edsurveyTable(formula = ~ x_chsex_r + p9curmar, data = eclsk11, weightVar = "w9c29p_9t90", varM
```

```
## Warning in calcEdsurveyTable(formula, data, weightVar, jrrIMax,
## pctAggregationLevel, : Removing 2251 rows with 0 weight from analysis.
```

```
tbl1
```

```
##
## Formula: childid ~ x_chsex_r + p9curmar
##
## Weight variable: 'w9c29p_9t90'
## Variance method: jackknife
## JK replicates: 80
## full data n: 18174
## n used: 7945
##
##
## Summary Table:
##    x_chsex_r                                p9curmar    N      WTD_N       PCT
## 1    1: MALE                          1: MARRIED (1) 2938 1367616.83 67.608642
## 2    1: MALE                        2: SEPARATED (2)  151   86412.02  4.271810
## 3    1: MALE           3: DIVORCED OR WIDOWED (3, 4)  442  250607.34 12.388866
## 4    1: MALE                    4: NEVER MARRIED (5)  425  273190.02 13.505250
## 5    1: MALE 5: CIVIL UNION/DOMESTIC PARTNERSHIP (6)   81   45017.01  2.225432
## 6  2: FEMALE                          1: MARRIED (1) 2870 1319848.64 69.131210
## 7  2: FEMALE                        2: SEPARATED (2)  143   80672.81  4.225491
## 8  2: FEMALE           3: DIVORCED OR WIDOWED (3, 4)  428  224738.15 11.771365
## 9  2: FEMALE                    4: NEVER MARRIED (5)  385  237346.10 12.431746
## 10 2: FEMALE 5: CIVIL UNION/DOMESTIC PARTNERSHIP (6)   82   46587.90  2.440187
##      SE(PCT)
## 1  1.1756039
## 2  0.3944507
## 3  0.7625198
## 4  0.9075561
## 5  0.3368478
## 6  1.0257652
## 7  0.4357400
## 8  0.6138104
## 9  0.7270084
## 10 0.2406026
```

The `edsurveyTable()` function also features multiple variance estimation methods. As shown in the previous example, the default `varMethod = "jackknife"` uses the **jackknife** method. By setting `varMethod = "Taylor"`, you can switch to the **Taylor series** method.

```r
tbl2 <- edsurveyTable(formula = ~ x_chsex_r + p9curmar, data = eclsk11, weightVar = "w9c29p_9t90", varM
```

```
## Warning in calcEdsurveyTable(formula, data, weightVar, jrrIMax,
## pctAggregationLevel, : Removing 2251 rows with 0 weight from analysis.
```

```r
tbl2
```

```
##
## Formula: childid ~ x_chsex_r + p9curmar
##
## Weight variable: 'w9c29p_9t90'
## Variance method: Taylor series
## full data n: 18174
## n used: 7945
##
##
## Summary Table:
##    x_chsex_r                                p9curmar    N      WTD_N       PCT
## 1    1: MALE                          1: MARRIED (1) 2938 1367616.83 67.608642
## 2    1: MALE                        2: SEPARATED (2)  151   86412.02  4.271810
## 3    1: MALE           3: DIVORCED OR WIDOWED (3, 4)  442  250607.34 12.388866
## 4    1: MALE                     4: NEVER MARRIED (5)  425  273190.02 13.505250
## 5    1: MALE 5: CIVIL UNION/DOMESTIC PARTNERSHIP (6)   81   45017.01  2.225432
## 6  2: FEMALE                          1: MARRIED (1) 2870 1319848.64 69.131210
## 7  2: FEMALE                        2: SEPARATED (2)  143   80672.81  4.225491
## 8  2: FEMALE           3: DIVORCED OR WIDOWED (3, 4)  428  224738.15 11.771365
## 9  2: FEMALE                     4: NEVER MARRIED (5)  385  237346.10 12.431746
## 10 2: FEMALE 5: CIVIL UNION/DOMESTIC PARTNERSHIP (6)   82   46587.90  2.440187
##      SE(PCT)
## 1  1.3241743
## 2  0.4490012
## 3  0.7819802
## 4  1.1456202
## 5  0.3459628
## 6  1.1836404
## 7  0.4377188
## 8  0.6806314
## 9  0.8638412
## 10 0.3446689
```

Note that the SE(PCT) values are slightly different (but all other values are identical) between the two estimation methods.

If the percentages do not add up 100 'at the desired level', an adjustment can be made in the `pctAggregationLevel` argument to change the aggregation level. The default `pctAggregationLevel = 1` aggregates by each level of the first variable – in our example, this is `x_chsex_r`. Setting `pctAggregationLevel = 0` aggregates by each level of each variable.

```r
tbl3 <- edsurveyTable(formula = ~ x_chsex_r + p9curmar, data = eclsk11, weightVar = "w9c29p_9t90", varM
                      pctAggregationLevel = 0)
```

```
## Warning in calcEdsurveyTable(formula, data, weightVar, jrrIMax,
## pctAggregationLevel, : Removing 2251 rows with 0 weight from analysis.
```

```r
tbl3
```

```
##
## Formula: childid ~ x_chsex_r + p9curmar
##
## Weight variable: 'w9c29p_9t90'
## Variance method: jackknife
## JK replicates: 80
## full data n: 18174
## n used: 7945
##
##
## Summary Table:
##    x_chsex_r                                p9curmar    N      WTD_N       PCT
## 1    1: MALE                          1: MARRIED (1) 2938 1367616.83 34.781384
## 2    1: MALE                        2: SEPARATED (2)  151   86412.02  2.197640
## 3    1: MALE           3: DIVORCED OR WIDOWED (3, 4)  442  250607.34  6.373474
## 4    1: MALE                     4: NEVER MARRIED (5)  425  273190.02  6.947799
## 5    1: MALE 5: CIVIL UNION/DOMESTIC PARTNERSHIP (6)   81   45017.01  1.144878
## 6  2: FEMALE                          1: MARRIED (1) 2870 1319848.64 33.566538
## 7  2: FEMALE                        2: SEPARATED (2)  143   80672.81  2.051680
## 8  2: FEMALE           3: DIVORCED OR WIDOWED (3, 4)  428  224738.15  5.715566
## 9  2: FEMALE                     4: NEVER MARRIED (5)  385  237346.10  6.036213
## 10 2: FEMALE 5: CIVIL UNION/DOMESTIC PARTNERSHIP (6)   82   46587.90  1.184829
##      SE(PCT)
## 1  0.6703083
## 2  0.2053153
## 3  0.4022999
## 4  0.4612130
## 5  0.1737175
## 6  0.5561966
## 7  0.2116686
## 8  0.3015585
## 9  0.3620376
## 10 0.1182617
```

Note the changes in the PCT values.

Subsetting Data

A subset of a data set can be used with the `{EdSurvey}` functions. In the following example, a summary table is created after filtering the sample to include only students who are female (`x_chsex_r` = "2: FEMALE") and White or Black (`x_raceth_r` = 1 [WHITE, NON-HISPANIC] or 2 [BLACK/AFRICAN AMERICAN, NON-HISPANIC]).

```r
dat2 <- subset(eclsk11, x_chsex_r %in% "2: FEMALE" & (x_raceth_r %in% 1 | x_raceth_r %in% 2))

tbl4 <- edsurveyTable(formula = ~ x_raceth_r, data = dat2, weightVar = "w9c29p_9t90")
```

```
## Warning in calcEdsurveyTable(formula, data, weightVar, jrrIMax,
## pctAggregationLevel, : Removing 2841 rows with 0 weight from analysis.
```

```r
tbl4
```

```
##
## Formula: childid ~ x_raceth_r
##
## Weight variable: 'w9c29p_9t90'
## Variance method: jackknife
## JK replicates: 80
## full data n: 18174
## n used: 2393
##
##
## Summary Table:
##                                x_raceth_r    N    WTD_N      PCT  SE(PCT)
##                    1: WHITE, NON-HISPANIC 2067 982255.1 79.32355 1.779226
##  2: BLACK/AFRICAN AMERICAN, NON-HISPANIC  326 256034.2 20.67645 1.779226
```

3.3. Linear Regression

The `lm.sdf()` function performs general linear modeling (GLM; e.g., linear regression) while fully accounting for the complex sampling design. For example,

```r
out1 <- lm.sdf(formula = x9mscalk5 ~ x12sesl + x_chsex_r, data = eclsk11, weightVar = "w9c29p_9t90")
```

```
## Warning in calc.lm.sdf(formula = formula, data = data, weightVar = weightVar, :
## Removing 2467 rows with nonpositive weight from analysis.
```

```r
summary(out1, src = TRUE)
```

```
##
## Formula: x9mscalk5 ~ x12sesl + x_chsex_r
##
## Weight variable: 'w9c29p_9t90'
## Variance method: jackknife
## JK replicates: 80
## full data n: 18174
## n used: 7877
##
## Coefficients:
##                          coef         se        t    dof   Pr(>|t|) stdCoef
## (Intercept)        121.697214   0.362917 335.3305 47.291 0.0000e+00      NA
## x12sesl              9.141740   0.283475  32.2488 52.998 0.0000e+00  0.4093
## x_chsex_r2: FEMALE  -2.171360   0.419933  -5.1707 38.322 7.6194e-06 -0.0622
##                      stdSE
## (Intercept)             NA
## x12sesl            0.01269 *
## x_chsex_r2: FEMALE 0.01203 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared: 0.1711
```

[Note] By default, the standardized coefficients (stdCoef) are calculated using standard deviations of the variables themselves. When `standardizeWithSamplingVar = TRUE` is set, the variance of the standardized coefficient is calculated similar to a regression coefficient and therefore includes the sampling variance in the variance estimate of the outcome variable.

3.4. MANCOVA

The `mvrlm.sdf()` function performs GLM with multiple dependent variables (e.g., multivariate analysis of covariance) In the following example, multivariate GLM is conducted with two dependent variables (`x9mscalk5`, `x9rscalk5`) and two independent variables (`x_chsex_r`, `p9hlphwk`). We will use the `dat1` data set we prepared earlier by recoding specific -1 values to 0 for the `p9hlphwk` variable to account for survey skip patterns.

```
out2 <- mvrlm.sdf(x9mscalk5 | x9rscalk5 ~ x_chsex_r + p9hlphwk, data = dat1, weightVar = "w9c29p_9t90")
```

```
## Warning in calc.mvrlm.sdf(formula = formula, data = data, weightVar =
## weightVar, : Removing rows with 0 weight from analysis.
```

```
summary(out2)
```

```
##
## Formula: x9mscalk5 | x9rscalk5 ~ x_chsex_r + p9hlphwk
##
## Weight variable: 'w9c29p_9t90'
## Variance method:
## JK replicates: 80
## full data n: 18174
## n used: 7902
##
## Coefficients:
##
## x9mscalk5
##                          coef        se         t    dof  Pr(>|t|)
## (Intercept)         129.48360   0.91514 141.49092 43.911 < 2.2e-16 ***
## x_chsex_r2: FEMALE   -2.13178   0.43307  -4.92250 38.010 1.690e-05 ***
## p9hlphwk             -2.77123   0.28759  -9.63617 55.219 1.987e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## x9rscalk5
##                          coef        se         t    dof  Pr(>|t|)
## (Intercept)         142.51179   0.74123 192.26519 67.227 < 2.2e-16 ***
## x_chsex_r2: FEMALE    1.43687   0.38947   3.68934 46.336 0.0005897 ***
## p9hlphwk             -2.12750   0.21480  -9.90446 54.691 8.359e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual correlation matrix:
##
##            x9mscalk5 x9rscalk5
## x9mscalk5      1.000     0.751
## x9rscalk5      0.751     1.000
```

```
##
## Multiple R-squared by dependent variable:
##
## x9mscalk5 x9rscalk5
##    0.0384    0.0289
```

Note that the vertical line symbol (|) separates dependent variables on the left-hand side of formula.

3.5. Logistic Regression

The `glm.sdf()` function performs logistic regression with a *binary* dependent variable while fully accounting for the complex sampling design. This function can also run probit regression, with the probit link function specified.

Although some dependent variables may already be binary, the function `I()` can be used in the formula to specify the desired outcome level for a non-binary variable. The following example demonstrates how to predict whether a student participates in an arts program (`p9perfrm`) based on their sex (`x_chsex_r`). The desired outcome level ("Yes") is specified as such `I(p9perfrm == 1)`.

```
out3 <- glm.sdf(I(p9perfrm == 1) ~ x_chsex_r, family = binomial(link = "logit"), data = eclsk11, weight
```

```
## Warning in calc.glm.sdf(formula = formula, family = family, data = data, :
## Removing 2096 rows with nonpositive weight from analysis.
```

```
## Warning in getStratumVar(data): The data may have more than one weight variable
## without a default. Use the 'showWeights' function to populate the 'weightVar'
## argument.
```

```
summary(out3)
```

```
##
## Formula: p9perfrm ~ x_chsex_r
## Family: binomial (logit)
##
## Weight variable: 'w9c29p_9t90'
## Variance method: jackknife
## JK replicates: 80
## full data n: 18174
## n used: 7552
##
## Coefficients:
##                          coef         se          t    dof  Pr(>|t|)
## (Intercept)         -1.921820   0.067824 -28.335533 49.136 < 2.2e-16 ***
## x_chsex_r2: FEMALE   1.278549   0.078918  16.201046 57.840 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The "log" odds of participating in an arts program increases by 1.28 for female students compared to male students. Logistic regression results can be further (and more easily) interpreted with **odds ratio** and **Wald test** results.

3.5.1. Odds Ratio

The `oddsRatio()` helper function converts the coefficients from an `{EdSurvey}` logistic regression model to odds ratios. Odds ratios are useful for understanding the real likelihood of an event occurring.

```
oddsRatio(out3)
```

```
##                           OR      2.5%     97.5%
## (Intercept)        0.1463404 0.1276955 0.1677077
## x_chsex_r2: FEMALE 3.5914247 3.0665979 4.2060719
```

The odds of participating in an arts program increases by 3.59 for female students compared to male students. In other words, female students are 3.59 times more likely to participate in an arts program.

3.5.2. Wald Test

The `waldTest()` function performs Wald test. The 2nd coefficient in our logistic model is tested in the following example:

```
waldTest(model = out3, coefficients = 2)
```

```
## Wald test:
## ----------
## H0:
## x_chsex_r2: FEMALE = 0
##
## Chi-square test:
## X2 = 262.5, df = 1, P(> X2) = 0.0
##
## F test:
## W = 262.5, df1 = 1, df2 = 70, P(> W) = 0
```

4. Summary

This lecture note covers the basics of the `{EdSurvey}` package, such as setting up the R environment, exploring variable distributions with `summary2()`, retrieving data for manipulation, creating summary tables with `edsurveyTable()`, correlating variables with `cor.sdf()`, conduct general linear modeling and MANCOVA with `lm.sdf()`, and fit logistic regression models with `glm.sdf()`. For a full list of the `{EdSurvey}` functions and documentation, use the R help viewer:

```
help(package = "EdSurvey")
```